

Python Program for Automation and Data Acquisition of Cavity Fourier-Transformed Microwave Spectrometer

Here we present a Python user interface designed to facilitate the control and operation of the custom-built cavity Fourier Transform Microwave (FTMW) setup within the L-shaped microwave spectrometer at Tennessee Tech. The cavity microwave setup, which consists of a Fabry-Perot resonant cavity and a microwave circuit, is coupled with a molecular beam source aiming to record rotational spectra of rotationally cold species. Additionally, a supersonically expanding molecular beam environment facilitates generating stable radicals and van der Waals complexes, making it an ideal platform for investigating the cold chemistry of astrochemically relevant species. Recently, we created a Python user interface for this home-built setup, and this is the focus of this presentation.

The custom Python script communicates with four devices in the FTMW setup: a Valon Synthesizer, an SRS Pulse Delay Generator, a Tektronix Oscilloscope, and a Zaber XNA08A25-E09 actuator. The Valon Synthesizer, controlled via USB and Serial API through `valonController.py`, generates microwaves with commands converted to UTF-8. The SRS Pulse Delay Generator sets the trigger rate and controls multiple devices via GPIB and pyVISA API through `SRSController.py`, with crucial timing managed by the `startTrig()` function. The Tektronix Oscilloscope, controlled via GPIB and pyVISA through `oscilloscopeController.py`, digitizes analog microwave signals with various data acquisition methods stored in numpy arrays. The Zaber actuator, managed via USB and ASCII through `zaberController.py`, precisely controls the Fabry-Perot cavity length. Performance is optimized using the Time API for function speed checks and the Threading API for concurrent data acquisition and GUI management, achieving speeds of over 450 data points per second. Data analysis and plotting are handled by NumPY and Matplotlib, with position and intensity data synchronized to high accuracy.

Device Control and Functionality:

1. Valon Synthesizer:

- **Functionality:** Generates microwaves.
- **Connection:** Controlled via USB through a Serial API.
- **Initialization:** `valonController.py` initializes the Valon synthesizer via a Python object.
- **Commands:** Uses `writeValonCommand()` to convert string commands to UTF-8 for altering settings directly from the software.

2. SRS Pulse Delay Generator:

- **Functionality:** Sets the trigger rate and controls multiple devices (oscilloscope, waveform generator).
- **Connection:** Controlled via GPIB through pyVISA API.
- **Initialization:** `SRSController.py` initializes the SRS pulse delay generator.
- **Commands:** pyVISA API converts string commands to binary (UTF-8) and sends them to the delay generator. The `startTrig()` function is crucial for initiating

triggering sequences, managing timing for waveform generation, data acquisition, and gas pulse repetition rate.

3. **Tektronix Oscilloscope:**

- **Functionality:** Receives analog microwave signals and converts them to digital for interpretation.
- **Connection:** Controlled via GPIB through pyVISA API.
- **Initialization:** oscilloscopeController.py initializes the oscilloscope.
- **Data Acquisition:** Acquires various types of data (waveforms, Fourier transformed signals, etc.) with commands differing for each acquisition method. Binary waveform data is stored in NumPY arrays, and FFT can be performed either on the oscilloscope or through additional code.

4. **Zaber XNA08A25-E09 Actuator:**

- **Functionality:** Controls the length of the Fabry-Perot cavity with high precision.
- **Connection:** Controlled via USB as an ASCII device through Serial API and Zaber API.
- **Initialization:** zaberController.py initializes and establishes settings for the Zaber actuator.
- **Commands:** A variety of functions manipulate position and movement velocity. Custom commands are sent via ASCII Serial connection through formatted strings, allowing for fine-tuned control with high sensitivity and small step sizes. This precision is crucial for maintaining the integrity of the Fabry-Perot cavity.

Program Workflow and Optimization:

The program ensures efficient and synchronized operation of the FTMW setup. The following steps outline the workflow.

1. **Initialization:**

- The Python script establishes connections with each device and confirms their responsiveness.
- Preliminary settings are initialized to prepare the devices for the experiment.

2. **Parameter Configuration:**

- User-provided parametric settings are configured to meet specific experimental requirements.

3. **Experiment Initiation:**

- The experiment is initiated by triggering the devices.

4. **Data Collection and Processing:**

- The script collects output data from the devices in real time, processing and analyzing it according to the experiment's goals.

5. **Performance Optimization:**

- The Time API is used to check performance and data acquisition speed by testing the time length of functions, ensuring fast acquisition.
- The Threading API is employed to acquire waveform data, host the Tkinter GUI, and alter other components according to data output (e.g., Zaber position)

simultaneously via separate threads at speeds of over 450 data points per second.

6. Data Analysis and Visualization:

- NumPY is used for complex matrix operations, generating linearly spaced arrays, and handling N-dimensional arrays.
- Matplotlib is used for plotting acquired data, ensuring synchronization of position data versus intensity data handled via time modules with an accuracy of ± 0.007 seconds.

Automation and Efficiency:

The comprehensive automation of the FTMW setup enhances the spectrometer's efficiency, making it more user-friendly and streamlined for operators. The program communicates with each device to initialize, configure settings, and capture real-time data for microwave generation, data acquisition, cavity-mirror movement, and timing control. This ensures precise and accurate measurements, critical for investigating the cold chemistry of astrochemically relevant species, providing a robust platform for further research in molecular spectroscopy and astrochemistry.